50 SQL Lab Exercises

MySQL & PostgreSQL | Beginner Practical Workbook

Topics covered: database and table creation, INSERT, SELECT, WHERE, LIKE, relational operators, logical operators, LIMIT, and ORDER BY.

How to use this workbook

• Create the database and tables shown below before attempting the exercises.

• Write and execute each query in both MySQL and PostgreSQL wherever possible.

• Use single quotes for text values. End each SQL statement with a semicolon.

• For PostgreSQL, use LIMIT in the same way as MySQL. The core exercises avoid database-specific features.

• Unless a question says otherwise, do not use SELECT *; select only the requested columns.

Part 1 – Create the practice database and tables

Database: Create a database named `sql_lab` (in PostgreSQL, connect to the database after creating it).

Table 1: students

```
student_id INT PRIMARY KEY
student_name VARCHAR(50) NOT NULL
age INT
gender VARCHAR(10)
city VARCHAR(40)
course VARCHAR(40)
marks INT
email VARCHAR(100)
```

Table 2: employees

```
employee_id INT PRIMARY KEY
employee_name VARCHAR(50) NOT NULL
department VARCHAR(40)
job_title VARCHAR(40)
salary DECIMAL(10,2)
city VARCHAR(40)
experience_years INT
```

Table 3: products

```
product_id INT PRIMARY KEY
product_name VARCHAR(60) NOT NULL
category VARCHAR(40)
price DECIMAL(10,2)
stock INT
brand VARCHAR(40)
```

Note: Choose suitable data types and constraints. You may add sample records of your own or use the values implied by the questions. Ensure that your sample data includes different cities, courses, departments, salaries, product categories, prices, and stock quantities so that filters return meaningful results.

Part 2 – Database and table creation (Exercises 1–8)

1. Create a database named **sql_lab**.

2. Select or connect to the **sql_lab** database.

3. Create the **students** table with all the columns and data types specified in Part 1.

4. Create the **employees** table with all the columns and data types specified in Part 1.

5. Create the **products** table with all the columns and data types specified in Part 1.

6. Display the list of tables available in your database using the appropriate command for your DBMS.

7. Display the structure or column definitions of the **students** table.

8. Create a table named **departments** with department_id as an integer primary key and department_name as a VARCHAR(40) column.

Part 3 – INSERT operations (Exercises 9–16)

9. Insert at least 10 student records into the **students** table.

10. Insert at least 8 employee records into the **employees** table.

11. Insert at least 10 product records into the **products** table.

12. Insert one student record by specifying values for every column.

13. Insert a student record with a new student_id, name, age, city, course, marks, and email. Use a suitable gender value.

14. Insert three product records using a single INSERT statement with multiple VALUES groups.

15. Insert a student record whose city is **Calicut** and whose course is **Python**.

16. Try inserting a second student record with an existing student_id. Observe the primary-key constraint error and explain why it occurs.

Part 4 – SELECT operations (Exercises 17–23)

17. Display all columns and records from the **students** table.

18. Display only student_name, course, and marks from the **students** table.

19. Display employee_name, department, and salary from the **employees** table.

20. Display product_name, category, and price from the **products** table.

21. Display only the student records whose city is **Chennai**.

22. Display the names and email addresses of all students.

23. Display the names of all employees and their job titles.

Part 5 – WHERE and relational operators (Exercises 24–32)

24. Display students whose marks are greater than 80.

25. Display students whose marks are greater than or equal to 60.

26. Display students whose age is less than 21.

27. Display employees whose salary is greater than 30000.

28. Display employees whose experience_years is less than or equal to 3.

29. Display products whose price is equal to 500.

30. Display products whose stock is not equal to 0.

31. Display students whose course is not equal to **Java**.

32. Display employees whose salary is between 25000 and 50000 using relational comparison conditions (do not use BETWEEN).

Part 6 – Logical operators (Exercises 33–39)

33. Display students who are enrolled in **Python** AND have marks greater than 75.

34. Display students who live in **Calicut** OR **Chennai**.

35. Display employees who work in the **IT** department AND have more than 2 years of experience.

36. Display employees whose department is **HR** OR whose salary is greater than 45000.

37. Display products whose category is **Electronics** AND whose stock is greater than 5.

38. Display students whose marks are greater than 80 AND whose age is less than 23.

39. Display employees who are NOT working in the **Sales** department.

Part 7 – LIKE operator (Exercises 40–44)

40. Display students whose names start with the letter **A**.

41. Display students whose names end with the letter **a**.

42. Display students whose names contain the letters **an**.

43. Display employees whose names have exactly five characters.

44. Display products whose product_name contains the word or character sequence **phone**.

Part 8 – ORDER BY and LIMIT (Exercises 45–50)

45. Display all students sorted by marks in ascending order.

46. Display all students sorted by marks in descending order.

47. Display employees sorted by salary from highest to lowest.

48. Display products sorted by price in ascending order; if two products have the same price, sort those by product_name alphabetically.

49. Display the five students with the highest marks using ORDER BY and LIMIT.

50. Display the three most expensive products using ORDER BY and LIMIT.

Submission checklist

■ All three main tables are created successfully.

■ Sample records have been inserted into each table.

■ All 50 exercises have been executed and tested.

■ Queries that use filters have been tested with records that both match and do not match.

■ MySQL and PostgreSQL differences, if any, have been noted.

Instructor note: This workbook intentionally focuses on the SQL topics already covered. It does not require JOIN, GROUP BY, aggregate functions, subqueries, UPDATE, or DELETE.